

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Github Repository: https://github.com/Hasini923/Deep_Learning/tree/main/AutoEncoders

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

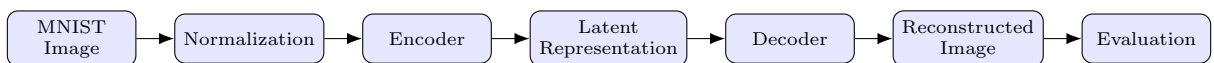
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

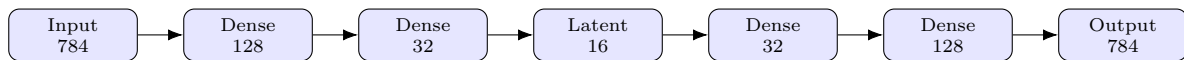
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

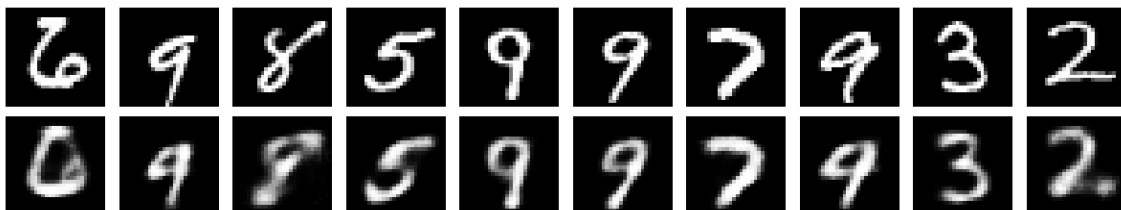
Plot 1: Original versus Reconstructed Images

Display at least 10 test images in a grid.

For every image, show:

Original | Reconstruction.

FC-AE: Original vs Reconstruction



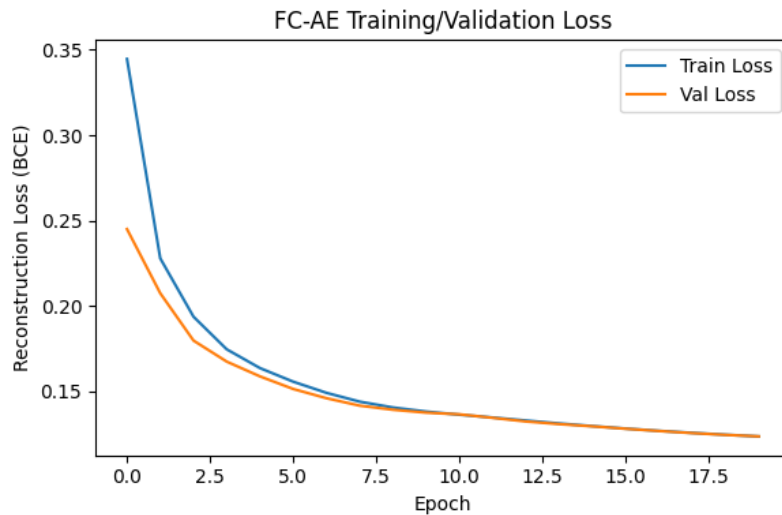
Students must comment on:

- preservation of digit shape;
- loss of fine details;
- blurred regions;
- differences between easy and difficult samples.

Inference: The FC-AE keeps the overall shape of every digit right, but it smudges the sharp edges and thin strokes, so the reconstructions look slightly blurry compared to the originals – a simple sign that flattening the image into a vector throws away some fine spatial detail.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss



Required inference: Determine whether the model converges and whether there is evidence of overfitting.

Inference: Both curves drop quickly at first and then flatten out around epoch 15–20, and the training and validation lines stay almost on top of each other the whole time – so the model converges smoothly and shows no sign of overfitting.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

Report:

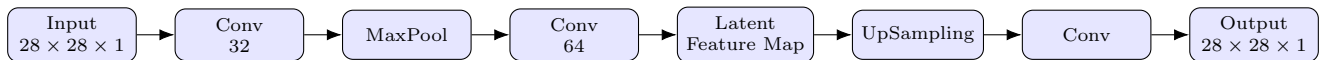
Metric	Value
Test MSE	0.020223
Test MAE	0.055450
Mean SSIM	0.7682

Inference: An SSIM close to 0.77 with a low MSE means the FC-AE reconstructions are structurally close to the originals but not pixel-perfect – basically ”good but blurry,” which matches what Plot 1 shows visually.

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```

Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
  
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

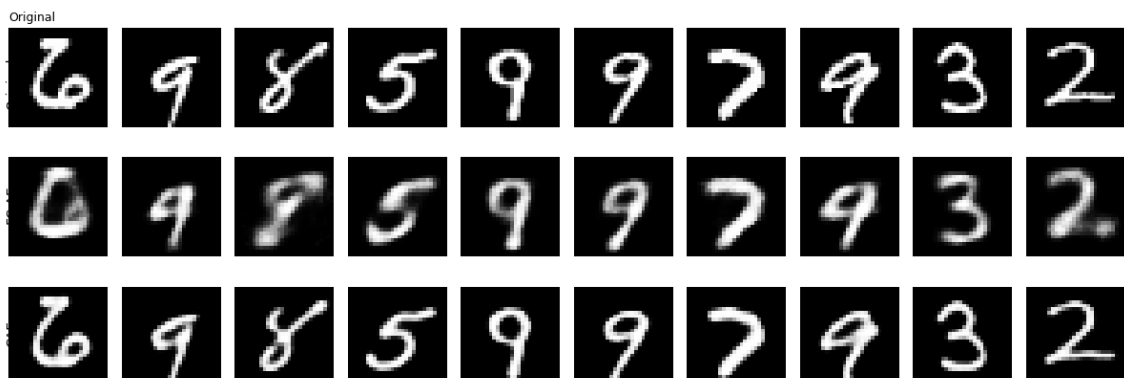
Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.020223	0.055450	0.7682	211,040
Convolutional AE	0.002522	0.015225	0.9739	74,497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.



Required inference:

Discuss how preserving spatial locality through convolution affects image reconstruction.

Inference: The Conv-AE reconstructions look noticeably sharper than the FC-AE ones and its MSE is almost 8x lower with fewer parameters – because convolution keeps each pixel’s neighbourhood intact instead of flattening the image, it reconstructs strokes and edges far more faithfully.

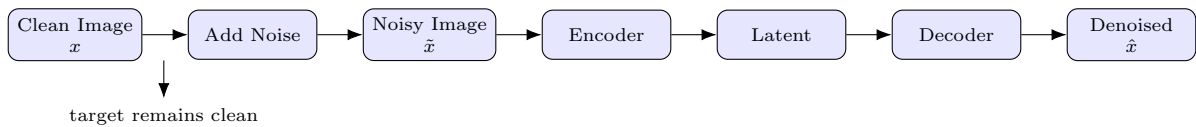
12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

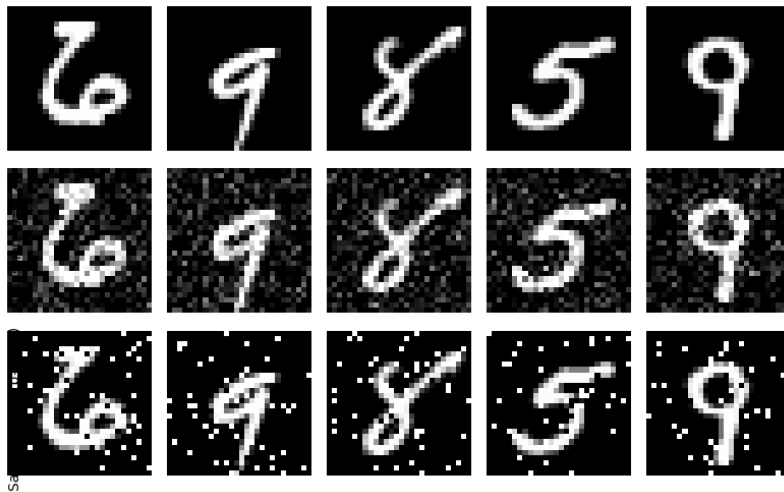
Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$



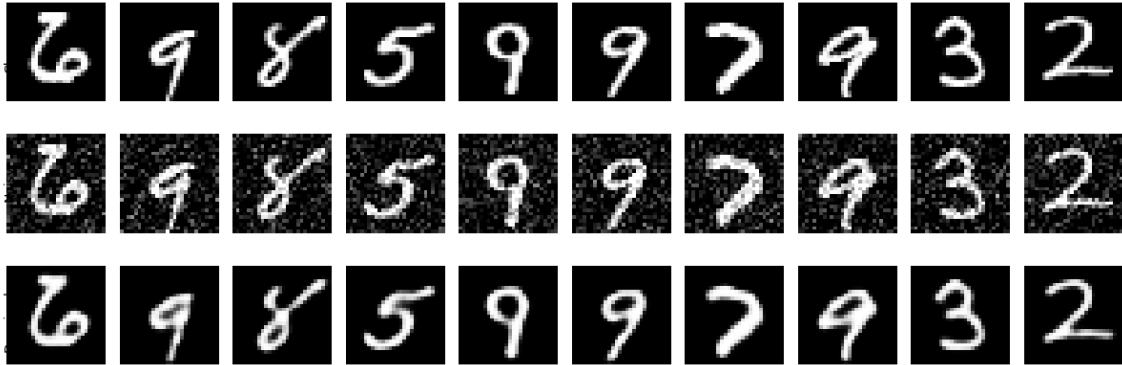
Inference: Gaussian noise spreads a light, grainy haze over the whole digit, while salt-and-pepper noise punches in sharp black/white specks – visually very different corruptions, which is why both are worth testing on the same denoising model.

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.



Inference: Even though the noisy row looks quite grainy, the denoised row comes out almost as clean as the original digits – the model has clearly learned to recognise digit structure and ignore random pixel noise.

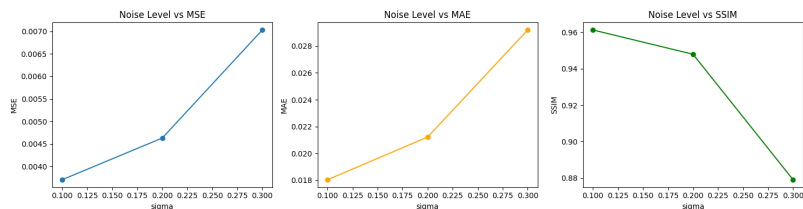
Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

MSE , MAE , $SSIM$.

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.003705	0.018036	0.9613
0.2	0.004630	0.021213	0.9480
0.3	0.007031	0.029196	0.8791



Required inference:

Explain how increasing corruption affects reconstruction quality and identify whether the denoising model is able to recover the major structure of the original image.

Inference: As the noise level climbs from 0.1 to 0.3, MSE and MAE both rise while SSIM steadily drops – so heavier corruption does make denoising harder, but even at the highest noise level SSIM stays close to 0.88, meaning the model still recovers the digit’s main structure well.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

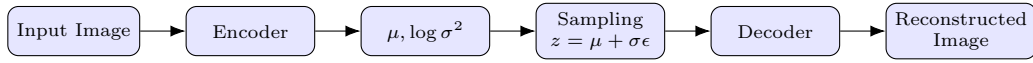
$$q_{\phi}(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

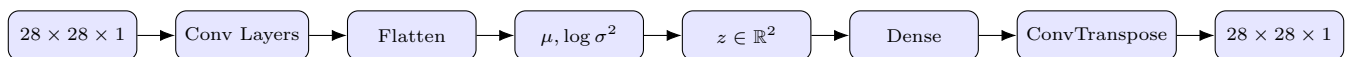
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

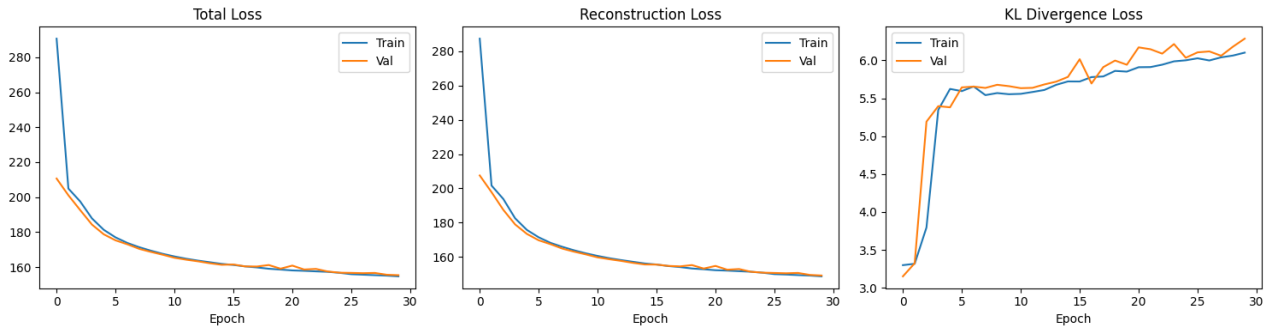
$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



Plot 9 (loss curve): VAE Training/Validation Loss

After training the VAE for 30 epochs, plot the total, reconstruction and KL-divergence loss curves.



Inference: The total and reconstruction losses fall steeply for the first few epochs and then flatten out, while the KL loss rises slightly before settling – this is the expected push-and-pull of a VAE, where the model first focuses on reconstructing well and then gradually regularises its latent space towards the standard normal prior.

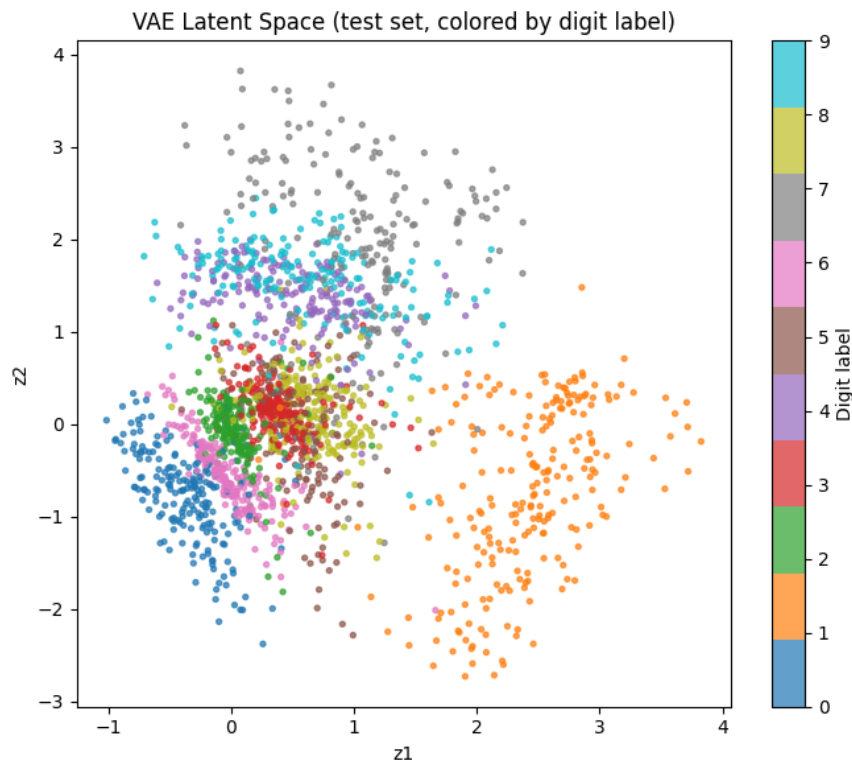
18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.



Required inference:

Students should describe whether samples belonging to the same digit occupy similar regions and whether different digits overlap.

Inference: Same-digit points mostly cluster together in their own patch of the 2-D plane, but neighbouring clusters (like the ones for visually similar digits) blend into each other at the edges – showing the VAE has learned a meaningful but not perfectly separated latent space.

19. Generating New Images with the VAE

Sample points from

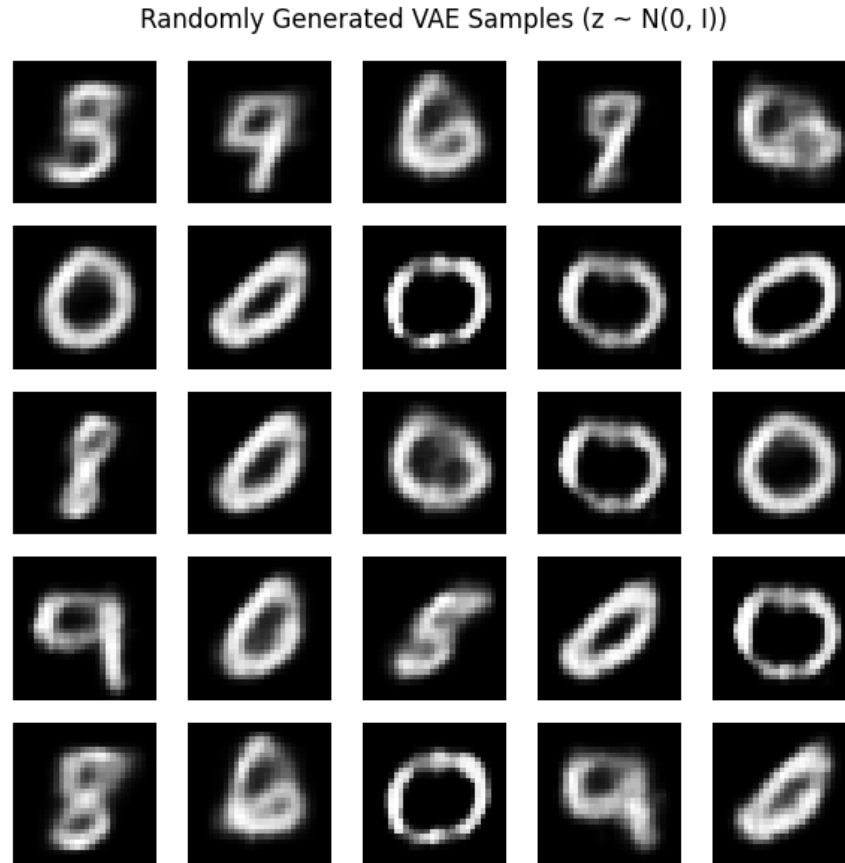
$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.



Students should determine:

- whether the generated images resemble handwritten digits;
- whether digit shapes are recognizable;
- whether there is visible variation among generated samples; and
- whether some samples are ambiguous or unrealistic.

Inference: Many of the generated samples come out as loop-like shapes (0s and round 6/8/9 forms) with only occasional sharper digits like 5 or 9, and a few blobs look blurry or ambiguous – because with only a 2-D latent space, sampling lands more often in the crowded, overlapping middle region of the space than in the smaller pockets reserved for less common digit shapes.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

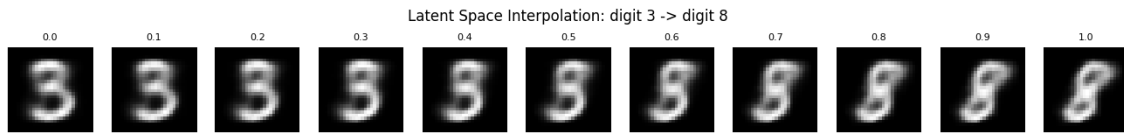
Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.



Required inference:

Describe whether the decoded image changes smoothly as the latent vector moves between two points.

Inference: Moving step by step from digit 3 to digit 8, the decoded images morph gradually with no sudden jumps – the shape slowly bends and closes up into the new digit, showing the latent space is smooth and continuous rather than made of disconnected islands.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.020223	0.055450	0.7682	211,040
Convolutional AE	0.002522	0.015225	0.9739	74,497
Denoising CAE	0.004652	0.021279	0.9477	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	149.052
KL Loss	6.285
Total Loss	155.338
Test MSE	0.043453
Test MAE	0.101844
Mean SSIM	0.5177

Inference: The Conv-AE gives the best reconstruction of all four models (lowest MSE, highest SSIM) with the fewest parameters, the Denoising CAE stays nearly as good despite being trained on corrupted inputs, and the VAE trails behind on pure reconstruction accuracy – which makes sense since part of its capacity is spent satisfying the KL constraint rather than only minimising pixel error.

22. Required Plots

The final report must contain the following plots:

1. Original image versus reconstructed image.
2. Training and validation loss for the basic autoencoder.
3. Fully connected AE versus convolutional AE reconstruction.
4. Clean versus noisy versus denoised images.
5. Noise level versus MSE/MAE/SSIM.
6. VAE latent-space visualization.
7. Randomly generated VAE images.
8. Latent-space interpolation.
9. Training/validation reconstruction loss for the VAE.
10. Reconstruction-error distribution on the test set.

For plots containing multiple quantitative measures, use separate figures or clearly labeled axes rather than placing incompatible scales on one axis.

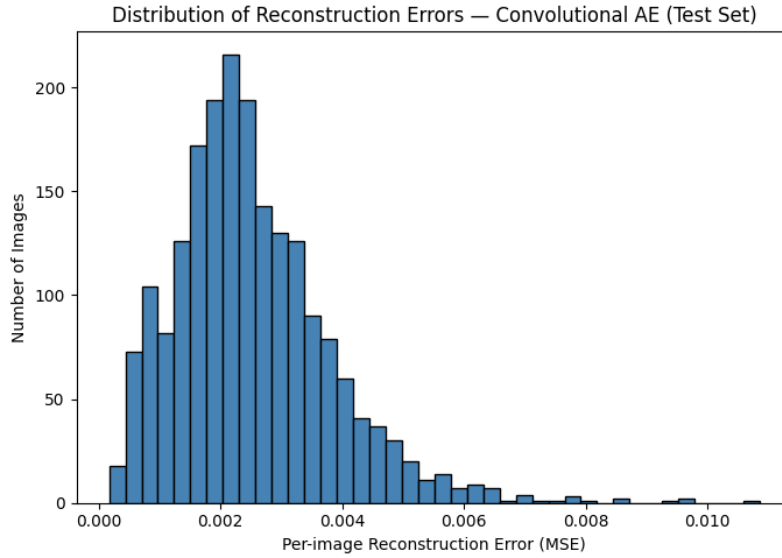
23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.



Error range: [0.000170, 0.010863], Mean = 0.002522, Median = 0.002334, Std = 0.001293 (computed on the Convolutional Autoencoder reconstructions).

Students should identify:

- the typical reconstruction-error range;
- whether the distribution is concentrated or spread out;
- possible high-error samples; and
- representative images corresponding to high reconstruction error.

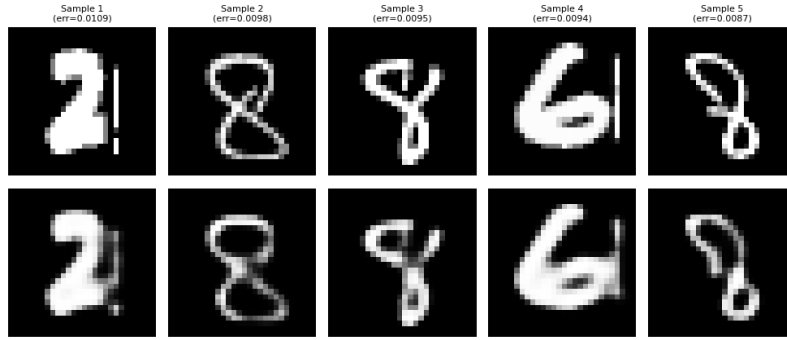
Inference: Most test images have a very small reconstruction error tightly bunched near the mean, with only a short tail of harder images stretching out to higher error values – so the model handles the vast majority of digits well, and only a handful of unusual samples are noticeably harder to reconstruct.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Sample	Test Index	True Label	Reconstruction Error
1	1670	2	0.010863
2	262	8	0.009761
3	417	8	0.009534
4	97	6	0.009394
5	1893	8	0.008697



Students should provide a short explanation for why these images may be difficult to reconstruct.

Inference: The worst-reconstructed digits are mostly unusually written 8s (three of the five) along with an oddly shaped 2 and 6 – these are digits with rarer stroke patterns or overlapping loops, so the model has seen fewer similar examples and reconstructs them less confidently.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.020223	0.055450	0.7682	211,040	12.66
Conv. Autoencoder	0.002522	0.015225	0.9739	74,497	22.94
Denoising CAE	0.004652	0.021279	0.9477	74,497	22.91
VAE	0.043453	0.101844	0.5177	184,421	32.55

The values must be obtained from the student's own execution.

Inference: The Convolutional Autoencoder is the best all-round performer here – best reconstruction quality, fewest parameters – while the VAE trades off some reconstruction accuracy and extra training time in exchange for a smooth, sample-able latent space that plain autoencoders don't have.

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;
- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

27. Additional Latent-Dimension Study

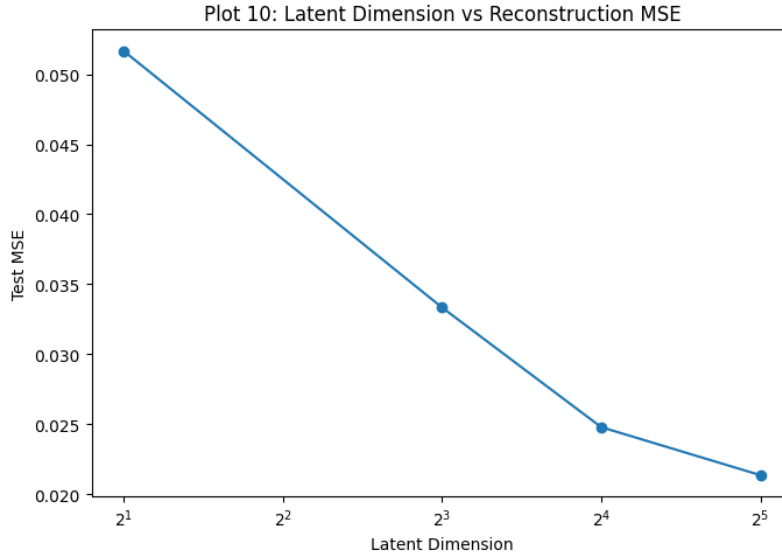
Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.051675	0.4158	210,130
8	0.033341	0.6413	210,520
16	0.024790	0.7164	211,040
32	0.021358	0.7564	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE



Students should explain the trade-off between compression and reconstruction quality.

Inference: MSE keeps dropping and SSIM keeps rising as the latent dimension grows from 2 to 32, but the gains get smaller each time (the jump from 2 to 8 helps far more than the jump from 16 to 32) – so a bigger latent code does mean better reconstruction, but with quickly diminishing returns, while the parameter count barely changes at all.

28. Discussion Questions

1. What is an autoencoder?
2. What is the purpose of the encoder?
3. What is the purpose of the decoder?
4. What is a latent representation?
5. Why is the latent representation usually smaller than the input?
6. What happens when the latent dimension is too small?
7. What happens when the latent dimension is increased?
8. Why can a convolutional autoencoder preserve spatial information more effectively?
9. Differentiate an autoencoder and a denoising autoencoder.
10. Why is the clean image used as the target in denoising?
11. What happens when the noise level is increased?
12. Why are MSE, MAE and SSIM useful for image reconstruction?
13. What is a Variational Autoencoder?
14. How does a VAE differ from a conventional autoencoder?
15. Why does a VAE learn a distribution instead of a single latent vector?
16. What is the reparameterization trick?
17. Why is ϵ sampled from a standard normal distribution?
18. What is the role of KL divergence in the VAE loss?
19. What is the prior distribution used in the VAE?
20. Why is a two-dimensional latent space useful for visualization?
21. What does latent-space interpolation demonstrate?
22. Why can a VAE generate new images?
23. Why might some VAE-generated images be unclear?
24. What is the difference between reconstruction and generation?
25. Why should test images remain unseen during training?

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

30. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image $\rightarrow$ Encoder $\rightarrow$ Latent Representation $\rightarrow$ Decoder $\rightarrow$ Reconstructed Image

They should also demonstrate denoising:

Clean Image $\rightarrow$ Noise $\rightarrow$ Denoising Autoencoder $\rightarrow$ Recovered Image

and variational generation:

Image $\rightarrow (\mu, \sigma) \rightarrow z \rightarrow$ Decoder $\rightarrow$ Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

31. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>